# 基础知识七：从日志理解 QKV 与 Gate 的维度

以 Qwen3.5-0.8B-Base 的真实请求为例，只解释输入形状，以及配置如何决定 Q、K、V 和输出门控(Gate)的维度。条件是单卡、张量并行(Tensor Parallelism，TP)=1，观察全注意力(Full Attention)层。

## 1. 完整原始配置

完整保留用户提供的配置字段和值，仅合并部分换行。下面的维度计算使用 `text_config` 中的参数。


```
{
    "architectures": ["Qwen3_5ForConditionalGeneration"],
    "image_token_id": 248056, "model_type": "qwen3_5",
    "text_config": {
        "attention_bias": false, "attention_dropout": 0.0,
        "attn_output_gate": true,
        "dtype": "bfloat16", "eos_token_id": 248044,
        "full_attention_interval": 4,
        "head_dim": 256,
        "hidden_act": "silu",
        "hidden_size": 1024,
        "initializer_range": 0.02, "intermediate_size": 3584,
        "layer_types": [
            "linear_attention", "linear_attention", "linear_attention", "full_attention",
            "linear_attention", "linear_attention", "linear_attention", "full_attention",
            "linear_attention", "linear_attention", "linear_attention", "full_attention",
            "linear_attention", "linear_attention", "linear_attention", "full_attention",
            "linear_attention", "linear_attention", "linear_attention", "full_attention",
            "linear_attention", "linear_attention", "linear_attention", "full_attention"
        ],
        "linear_conv_kernel_dim": 4, "linear_key_head_dim": 128,
        "linear_num_key_heads": 16, "linear_num_value_heads": 16,
        "linear_value_head_dim": 128,
        "max_position_embeddings": 262144, "mlp_only_layers": [],
        "model_type": "qwen3_5_text",
        "mtp_num_hidden_layers": 1, "mtp_use_dedicated_embeddings": false,
        "num_attention_heads": 8,
        "num_hidden_layers": 24,
        "num_key_value_heads": 2,
        "rms_norm_eps": 1e-06, "tie_word_embeddings": true, "use_cache": true,
        "vocab_size": 248320, "mamba_ssm_dtype": "float32",
        "rope_parameters": {
            "mrope_interleaved": true, "mrope_section": [11, 11, 10],
            "rope_type": "default", "rope_theta": 10000000,
            "partial_rotary_factor": 0.25
        }
    },
    "tie_word_embeddings": true, "transformers_version": "4.57.0.dev0",
    "video_token_id": 248057,
    "vision_config": {
        "deepstack_visual_indexes": [], "depth": 12,
        "hidden_act": "gelu_pytorch_tanh", "hidden_size": 768,
        "in_channels": 3, "initializer_range": 0.02,
        "intermediate_size": 3072, "model_type": "qwen3_5",
        "num_heads": 12, "num_position_embeddings": 2304,
        "out_hidden_size": 1024, "patch_size": 16,
        "spatial_merge_size": 2, "temporal_patch_size": 2
    },
    "vision_end_token_id": 248054, "vision_start_token_id": 248053
}
```

## 2. 输入多个 token，为什么日志里只有 `[1,1024]`？

请求是 请你描述下中国，10个字。加入聊天模板的角色和特殊标记后，一共是 **20 个 token**。日志对应的处理过程如下：

```
① 预填充(Prefill)：处理完整输入
   input_ids.shape=[20] → 向量化(Embedding)的输出形状=[20,1024]
   → self_attention() 的 hidden_states.shape=[20,1024] → 预测第一个新 token

② 解码(Decode)：处理上一轮生成的一个 token
   input_ids.shape=[1] → 向量化(Embedding)的输出形状=[1,1024]
   → self_attention() 的 hidden_states.shape=[1,1024] → 预测下一个 token
```

上图方括号表示张量形状，不是元素值。`hidden_states.shape=[本轮计算的 token 数, hidden_size]`，`hidden_size=1024` 决定每个 token 的向量宽度。`batch_size=1` 表示本批有一个请求，不代表请求只有一个 token。

全注意力层此前 token 的 K/V 已保存在 **GPU 显存(GPU memory)** 中；Decode 仍会读取历史 K/V，无需重新计算整段输入的 K/V。

日志对照：Prefill 的 `[20,1024]`、Decode 的 `[1,1024]`。

## 3. QKV 投影的输出为什么是 `[1,5120]`？

这段 Decode 日志来自 `model.language_model.layers.3.self_attn.qkv_proj`，类型是 `QKVParallelLinear`。形状摘录如下：

```
input_.shape=torch.Size([1,1024]), output.shape=torch.Size([1,5120]), output_bias.shape=None
q.shape=torch.Size([1,2048]), k.shape=torch.Size([1,512])
v.shape=torch.Size([1,512]), gate.shape=torch.Size([1,2048])
```

`self.qkv_proj(hidden_states)` 一次投影(Projection)生成打包的 Q、K、V 和 Gate，之后再拆分。各部分宽度由配置决定：

| 部分 | 对应配置与宽度计算 | 本轮形状 |
|---|---|---|
| Q：查询(Query) | `num_attention_heads × head_dim = 8 × 256 = 2048` | `[1,2048]` |
| K：键(Key) | `num_key_value_heads × head_dim = 2 × 256 = 512` | `[1,512]` |
| V：值(Value) | 与 K 的头数、每头宽度相同，即 `2 × 256 = 512` | `[1,512]` |
| Gate：输出门控 | `attn_output_gate=true`，宽度与 Q 相同，即 `8 × 256 = 2048` | `[1,2048]` |

**总输出宽度 = Q + K + V + Gate = 2048 + 512 + 512 + 2048 = 5120。** Prefill 同样如此，只是本轮 token 数为 20，因此输出为 `[20,5120]`。

`head_dim=256` 明确指定每个头的宽度，不要求 `num_attention_heads × head_dim` 等于 `hidden_size`；投影可以改变向量宽度。

`output_bias.shape=None` 表示该日志中没有输出偏置(Bias) Tensor。配置里是 `attention_bias=false`；当前代码构造 `qkv_proj` 时也直接指定了 `bias=False`，并非在此处读取 `attention_bias` 字段。

日志对照：投影输入与输出、返回的 Q/K/V/Gate 形状。

## 4. 术语速查

| 术语 | 中文含义 | 简明英文释义 |
|---|---|---|
| Q / K / V：Query / Key / Value | 注意力中的查询、键、值向量 | Query, key and value vectors used in attention. |
| Head /hed/ | 注意力头；本例每头 256 维 | One attention head. |
| Gate /geɪt/ | 控制注意力输出的门控向量 | A vector that gates the attention output. |
| Projection /prəˈdʒekʃən/ | 投影，将输入向量映射到输出向量 | A mapping from input vectors to output vectors. |
| TP：Tensor Parallelism | 张量并行；本例为 1，不跨卡拆分 | Partitioning tensor computation across devices. |